DAT12-3

Pandas I

Manipulating tabular data in Python

From a raw CSV to a clean, summarized DataFrame.

Albert School — 16 hours

Preface

In your first programming course you learned to read a CSV file the hard way: open it with a context manager, loop over its lines, split each on a comma, and build up lists or dictionaries by hand. That works, and it taught you what a table really is — rows, columns, and the awkward business of keeping them aligned. But it does not scale. The moment you want the average of a column, the rows where one value exceeds another, or a summary broken down by category, the hand-written loops pile up and every one of them is a chance to make a quiet, uncheckable mistake.

This course hands you the professional's tool for that work: **pandas**. Pandas does not give you a new kind of data — a table is still a table, exactly the one you built by hand before. What it gives you is a **vocabulary** for operating on whole tables at once: select these rows, group by that column, join this table to that one, reshape wide into long. Where you once wrote a ten-line loop, you now write one line that says what you want, and pandas runs the loop for you, correctly, on a million rows as easily as on ten.

The skill this course builds is not memorising method names. It is learning to **see a data question as a sequence of table operations** — and then to read the result back as a sentence a human can act on. The same habit your spreadsheet course drilled — look at the data before you touch it — survives intact here; pandas only makes the looking and the touching faster. By the end you will take a raw CSV, inspect it, clean it, summarise it, and write a short paragraph saying what it means. That full arc — from messy file to defended finding — is the whole point, and every chapter is a step along it.

A note on running the code. Pandas is a third-party library: install it into your virtual environment with `pip install pandas` (and `openpyxl` if you read Excel files), exactly as you learned to install any package. Then, by universal convention, import it under the alias `pd`:

```
import pandas as pd
```

Every code example below assumes this line has run.

Contents

1 DataFrames and Series

1.1 Two objects, one of them built from the other

Everything in pandas is built from two objects. Meet the smaller one first.

Definition 1 (Series) A **Series** is a one-dimensional array of values together with an **index** — a sequence of labels, one per value. It is, in effect, a single column with its rows named.

Definition 2 (DataFrame) A **DataFrame** is a two-dimensional table: a collection of Series that share one common index (the row labels) and are distinguished by column names. Selecting one column of a DataFrame gives you back a Series.

The relationship is the key idea, so make it concrete before going further.

Worked example — A DataFrame is columns stacked side by side. Suppose we have three people:

	name	age	city
0	Alice	34	Paris
1	Bob	28	Lyon
2	Carmen	41	Paris

The whole table is a DataFrame. Its leftmost column — `0, 1, 2`, in grey — is not data; it is the **index**, the shared row labels. The `age` column on its own, carrying that same index, is a Series:

```
0    34
1    28
2    41
Name: age, dtype: int64
```

Read the DataFrame as three Series (`name`, `age`, `city`) standing side by side, all lined up on the same index. That mental picture explains every operation that follows.

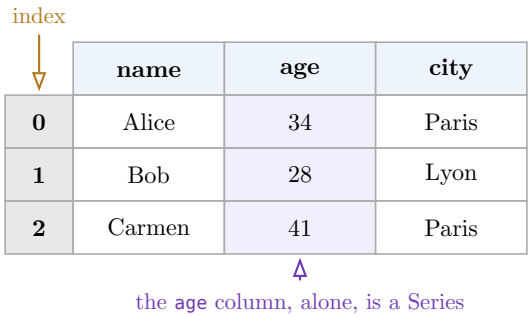

Figure 1: A DataFrame is several Series sharing one index. The grey index column holds the row labels; the highlighted `age` column, carried with that index, is itself a Series.

1.2 Loading data from a file

You rarely type a table by hand; you load it from a file. Pandas provides one reader per common format, each returning a DataFrame:

```python
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")    # needs the openpyxl package
df = pd.read_json("data.json")
```

`read_csv` is the workhorse and has many options; the two you will reach for first handle the most common surprises:

```python
df = pd.read_csv("data.csv", sep=";")          # European CSVs often use ;
df = pd.read_csv("data.csv", encoding="latin-1")  # if accents come out garbled
```

Common pitfall A CSV carries no type information — it is just text. Pandas **guesses** each column's type from its contents, and a single stray value derails the guess: one `"N/A"` typed into an otherwise numeric column makes pandas read the **entire** column as text, and every later sum or mean on it will fail or mislead. This is the same number-stored-as-text trap you met in spreadsheets, and the cure is the same: inspect the types immediately after loading, before computing anything.

1.3 Inspecting before you touch

The first thing to do with any freshly loaded DataFrame is **look at it** — not compute, look. Five tools answer the five questions you should always ask.

Definition 3 (The five inspection tools)

- `df.shape` — a tuple `(rows, columns)`: how big is the table?
- `df.head(n)` — the first `n` rows (default 5): what do the values look like?
- `df.dtypes` — the stored type of each column: is each column the type I expect?
- `df.info()` — column names, non-null counts, and dtypes together: where are values missing?
- `df.describe()` — count, mean, standard deviation, min, the three quartiles, and max of each numeric column: how is each number distributed?

Worked example — Reading describe out loud. Calling `df.describe()` on a sales table might print, for the `price` column, `count 980, mean 24.3, std 11.2, min 0, 25% 16, 50% 22, 75% 30, max 999`. Read it as sentences, not figures: "There are 980 prices (so 20 are missing, since the table has 1000 rows). A typical price is around €22 (the median, the 50% row), the bulk sit between €16 and €30, and **something is wrong at both ends** — a minimum of €0 and a maximum of €999 are almost certainly errors, not sales." The numbers did the looking; you did the interpreting. That last step is the skill.

Common pitfall `df.describe()` silently ignores every non-numeric column and every missing value. A clean-looking summary can hide that half your rows have no price at all — the `count` row is what reveals it. Always compare each column's `count` against `df.shape[0]`; a gap is missing data you have not dealt with yet.

Remark `head` shows the **top** of the file, which is often the most regular part. Pair it with `df.tail()` (the last rows) and, once you know `describe`, with a glance at the min and max: errors love to hide where you are not looking.

2 Selection and Filtering

Loading gives you the whole table; analysis almost always needs a **part** of it — some columns, some rows, or the rows meeting a condition. Pandas separates these cleanly, and the commonest beginner confusion comes from mixing up the tools, so we keep them distinct.

2.1 Selecting columns

A single column comes out as a Series; a list of columns comes out as a smaller DataFrame:

```
df["age"]               # one column  -> Series
df[["age", "city"]]     # several columns -> DataFrame
```

Common pitfall The double brackets are not a typo. `df["age"]` (single) returns a Series; `df[["age"]]` (a one-element **list** of names) returns a one-column DataFrame. They print almost identically but behave differently downstream — a Series and a DataFrame support different methods. When you mean "a table of these columns", use a list, even if it has one element.

2.2 Selecting rows: loc and iloc

To pick **rows**, pandas gives you two indexers that look alike and mean opposite things. Getting them straight now saves you hours later.

Definition 4 (loc and iloc)

- `df.loc[rows, cols]` selects by **label** — the index labels and column names.
- `df.iloc[rows, cols]` selects by **integer position** — the 0-based row and column numbers, regardless of the labels.

Each takes a row selector and an optional column selector, separated by a comma.

Worked example — The same cell, reached two ways. Take our three-person table, whose index happens to be `0, 1, 2`.

```
df.loc[1, "age"]      # label 1, column "age"      -> 28
df.iloc[1, 1]         # 2nd row, 2nd column          -> 28
```

Here both give `28` only because the labels coincide with the positions. Change the index — say, set `name` as the index — and they diverge sharply: `df.loc["Bob", "age"]` still finds Bob, but `df.iloc[1, 1]` finds whoever happens to sit in row position 1. **Use `loc` when you know the label, `iloc` when you know the position.**

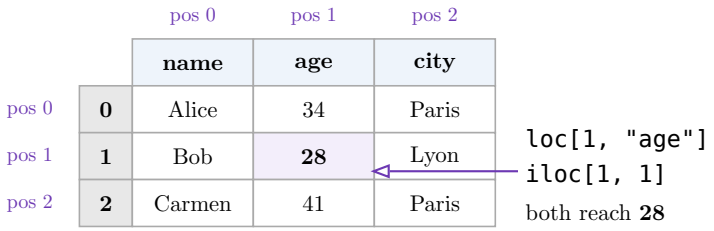

Figure 2: `loc` addresses by label (the names in the grey index and the column headers); `iloc` addresses by 0-based position (the violet "pos" guides). They agree here only because the index labels equal the positions.

Common pitfall With `loc`, a slice **includes** its right endpoint: `df.loc[0:5]` returns rows labelled `0` through `5` — six rows. With `iloc`, as everywhere else in Python, a slice **excludes** it: `df.iloc[0:5]` returns positions `0` through `4` — five rows. This off-by-one difference is one of the most common pandas bugs; the rule is "labels are inclusive, positions are exclusive."

2.3 Filtering with boolean indexing

The most powerful selection is by **condition**: keep the rows where something is true. The mechanism has two steps, and seeing them separately demystifies it.

Worked example — A condition is itself a column. Write a comparison on a column and pandas evaluates it for every row, producing a **boolean Series** — a column of `True`/`False`:

```
df["age"] > 30
# 0     True
# 1    False
# 2     True
```

Hand that boolean Series back to the DataFrame in brackets, and pandas keeps only the rows marked `True`:

```
df[df["age"] > 30]      # Alice and Carmen; Bob is dropped
```

So filtering is not a special command — it is computing a True/False column and using it as a mask. Once you see it that way, combining conditions is obvious.

Definition 5 (Combining conditions) Join boolean Series with `&` (and), `|` (or), and `~` (not). **Each condition must be wrapped in its own parentheses:**

```
df[(df["age"] > 30) & (df["city"] == "Paris")]
```

Common pitfall Two traps live in this one line. First, you must use `&` and `|`, **not** the Python keywords `and` and `or` — the keywords operate on single True/False values, not on whole columns, and raise an error here. Second, `&` and `|` bind more tightly than `>` and `==`, so without the inner parentheses `df["age"] > 30 & df["city"] == "Paris"` is grouped wrongly and fails. Always parenthesise each comparison.

3 Missing Values

Real data has holes. A respondent skipped a question, a sensor dropped a reading, a join found no match. Pandas marks every such hole with the special value `NaN` ("Not a Number"), and a large part of professional data work is deciding, column by column, what to do about them. The decision is yours to **justify**; pandas only supplies the tools.

3.1 Step 1 — Find them

You cannot handle what you cannot see, so locate the gaps first.

```
df.isna()              # a boolean DataFrame: True wherever a value is missing
df.isna().sum()        # missing count per column  (True counts as 1)
df.isna().mean()       # missing *fraction* per column
```

Worked example — Reading the missing-value profile. `df.isna().sum()` might return `age 20, price 0, city 145`. Now you know the shape of the problem before touching it: `price` is complete, `age` has a few gaps (20 of 1000 — minor), and `city` is missing for one row in seven — serious enough that how you treat it will visibly move any city-level summary. Different columns will need different decisions; this profile is what tells you so.

3.2 Step 2 — Decide: drop or impute

Once located, every missing value meets one of two fates: the row (or column) is **dropped**, or the gap is **filled** — **imputed** — with a substitute value.

```
df.dropna()                          # drop every row containing any NaN
df.dropna(subset=["price"])          # drop rows missing price specifically
df.dropna(axis=1)                    # drop every column containing any NaN
```

```
df["age"]  = df["age"].fillna(df["age"].median())   # numeric -> median or mean
df["city"] = df["city"].fillna("Unknown")           # categorical -> constant or mode
```

Definition 6 (Choosing a strategy)

- **Drop rows** when the missing values are few and scattered, so that losing those rows does not bias the result.
- **Drop a column** when it is missing so often that it carries little information.
- **Impute a numeric column** with its **mean** when the distribution is roughly symmetric, or its **median** when it is skewed or has outliers (the median resists them).
- **Impute a categorical column** with a constant such as `"Unknown"`, or with the **mode** (most frequent value) when a best guess is wanted.

Whichever you choose, **state it and why** — the choice changes the result and must be defensible.

Common pitfall Imputing is not free: filling 145 missing cities with `"Unknown"` invents a new, large category that will dominate a later `groupby("city")`; filling 200 missing ages with the mean shrinks the column's apparent spread and flatters its standard deviation. There is no neutral choice — dropping discards real rows, imputing fabricates values. Pick the lesser distortion **for the question you are answering**, and record it.

Common pitfall `df.dropna()` and `df.fillna()` return a **new** DataFrame and leave the original untouched. Writing `df.dropna()` on its own line does nothing lasting — you must reassign: `df = df.dropna()`. Forgetting the assignment, then wondering why the `NaN`s are still there, is a rite of passage; skip it.

4 Groupby and Aggregation

So far every operation has returned rows. The real analytical leap is **summarising**: collapsing many rows into one number per category — average sales **per city**, total revenue **per month**. Pandas expresses this with `groupby`, and the pattern behind it is worth naming because you will use it constantly.

Definition 7 (Split–apply–combine) `groupby` follows three steps. **Split** the rows into groups by the value of one or more columns; **apply** an aggregation (mean, sum, count, …) to a chosen column within each group, reducing it to one value; **combine** those per-group values into a new, smaller table indexed by the group.

Worked example — Average sales per city.

```
df.groupby("city")["sales"].mean()
```

Read left to right, this says: split the rows by `city`, take the `sales` column of each group, and average it. The result is one number per city:

```
city
Lyon      210.0
Paris     305.0
Name: sales, dtype: float64
```

Three thousand rows became two — and the two are the answer to "where do we sell more?" That compression, from rows to a finding, is what `groupby` is for.

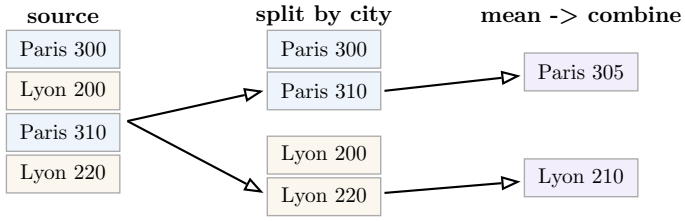

Figure 3: Split–apply–combine: rows are split into groups by `city`, the `mean` of `sales` is applied within each group, and the per-group results are combined into one row per city.

4.1 Aggregating several ways at once

`agg` lets one call compute different aggregations on different columns, or several on one:

```
df.groupby("city").agg({"sales": "sum", "age": "mean"})   # sum sales, mean age
df.groupby("city")["sales"].agg(["sum", "mean", "count"]) # three at once
```

4.2 Grouping by several dimensions

Pass a **list** of columns to summarise along more than one dimension at a time:

```
df.groupby(["city", "year"])["sales"].sum()
```

This produces one total per `(city, year)` pair — the table answer to "how did each city's sales evolve over the years?"

Common pitfall By default `groupby` returns a table **indexed by the grouping columns**, not an ordinary column you can select with `df["city"]`. To get a flat, normal DataFrame back — which most later steps expect — append `.reset_index()`:

```
df.groupby("city")["sales"].mean().reset_index()
```

Remark `groupby` skips `NaN` group keys entirely: rows whose `city` is missing simply vanish from a `groupby("city")` summary. If those rows matter, handle the missing values (previous chapter) **before** you group, or they will be silently dropped from your totals.

5 Merging and Concatenating

Data rarely arrives in one file. Customers live in one table, their orders in another; this year's sales sit beside last year's. Combining tables comes in two flavours, and confusing them is a classic error, so name the distinction up front: **concatenation** stacks tables that share a shape; **merging** joins tables that share a key.

5.1 Concatenation: stacking like with like

```
pd.concat([df1, df2])              # stack rows: more observations, same columns
pd.concat([df1, df2], axis=1)      # stack columns: more variables, same rows
```

Use `concat` when the pieces are **the same kind of thing** — January sales under February sales, all with identical columns.

5.2 Merging: joining on a key

> **Worked example — Attaching a price to each order.** An `orders` table lists a `product_id` per order but no price; a `products` table lists the price for each `product_id`. To put the price beside each order, you **merge** on the shared key:
>
> ```
> pd.merge(orders, products, on="product_id", how="left")
> ```
>
> Pandas matches each order's `product_id` to the product table and copies the price across. This is exactly the spreadsheet `VLOOKUP` you already know — now on whole tables at once, and able to fetch many columns in a single call.

The `how` argument decides **which rows survive when a key is missing on one side** — the single most important and most misunderstood choice in this chapter.

Definition 8 (The four join types) Merging tables `left` and `right` on a key:

- **inner** — keep only keys present in **both**. The safe default; no unmatched rows.
- **left** — keep **every** key of `left`; where `right` has no match, fill its columns with `NaN`.
- **right** — keep every key of `right`; symmetric to left.
- **outer** — keep every key of **either**; fill both sides' gaps with `NaN`.

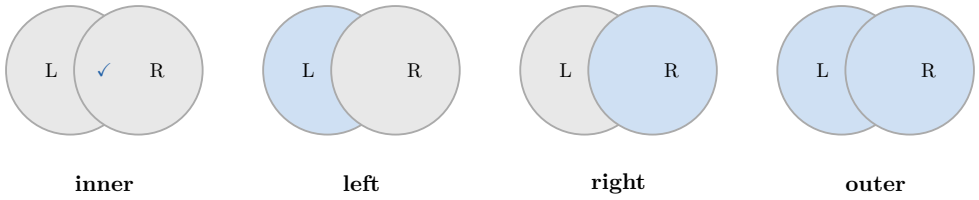

Figure 4: The four joins differ only in which keys are kept. Inner keeps the overlap; left keeps all of L; right keeps all of R; outer keeps everything. Shaded regions are retained.

> **Worked example — How the join type changes the answer.** Say `orders` has 1000 rows but 30 of them reference a `product_id` missing from `products` (a discontinued item).
>
> - `how="inner"` returns **970** rows — the 30 unmatched orders silently disappear.
> - `how="left"` returns all **1000** rows — the 30 keep their order data, with `NaN` for price.
>
> If your question is "total revenue", the inner join **undercounts** by dropping real orders; the left join keeps them visible (as `NaN` prices you must then handle). The join type is not a technicality — it decides which rows your analysis is even aware of.

Common pitfall After a merge, always check the row count. An inner join that quietly halves your rows means most keys did not match — often a sign of a dirty key (`"FR"` vs `"fr "`, integer vs string ids) rather than genuinely absent data. A merge that **grows** the row count beyond either input means the key was not unique on one side and rows were multiplied. Compare `len(result)` against your expectation every time.

6 Reshaping: Wide and Long

The same data can be laid out two ways, and many tasks are easy in one layout and painful in the other. Knowing how to switch between them is a quiet superpower.

Definition 9 (Wide and long format) In **wide** format, each variable has its own column and each entity is one row (a spreadsheet-style table). In **long** format, there is one row per **observation**, with columns naming the variable and holding its value. The same facts, reorganised.

Worked example — The same sales, two shapes. Wide — one column per city, easy for a human to read across:

date	Paris	Lyon
Jan	300	200
Feb	310	220

Long — one row per (date, city) observation, easy for a computer to group and plot:

date	city	sales
Jan	Paris	300
Jan	Lyon	200
Feb	Paris	310
Feb	Lyon	220

Both hold identical information. The wide form is better for reading and for a report; the long form is what `groupby`, merging, and most plotting libraries expect. You will move between them constantly.

Definition 10 (pivot and melt)

- `pivot` turns **long into wide**: it spreads the values of one column out into new columns.
- `melt` turns **wide into long**: it gathers a set of columns into two — one naming the old column, one holding its value.

```
# long  -> wide
wide = long.pivot(index="date", columns="city", values="sales")

# wide -> long
long = wide.melt(id_vars="date", var_name="city", value_name="sales")
```

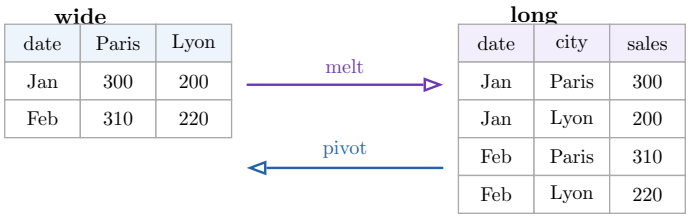

Figure 5: `melt` gathers the wide city columns into a single `city` column with a paired `sales` value; `pivot` spreads them back out. The information is unchanged — only its shape.

Common pitfall `pivot` fails if a single `(index, columns)` pair appears more than once — it does not know which value to keep and raises an error. When duplicates are expected (several sales on the same date in the same city), you want `pivot_table`, which **aggregates** the collisions (e.g. sums them) instead of erroring. Reach for `pivot` only when each combination is unique.

7 Applying Functions

Pandas' built-in operations cover most needs, but sometimes you must run **your own** logic on every value, column, or row. `apply` is the general escape hatch, and it pairs naturally with the `lambda` expressions you met in your Python course — small anonymous functions written inline.

Worked example — A custom transformation, element by element. Suppose prices are in euros and you want them in cents. There is no built-in "to cents", so you apply a function of your own:

```
df["cents"] = df["price"].apply(lambda x: round(x * 100))
```

Pandas calls the little function once per value of `price`, with `x` bound to that value, and collects the results into a new column. The `lambda` is just a one-line function: `lambda x: round(x * 100)` means "given `x`, return `round(x*100)`."

7.1 Across a column versus across a row

The `axis` argument decides whether your function sees one **value** at a time or a whole **row** at a time:

```
df["age"].apply(lambda x: x + 1)                    # Series: one value at a time
df.apply(lambda row: row["a"] + row["b"], axis=1)   # DataFrame, axis=1: whole row
```

With `axis=1`, the function receives an entire row (as a Series) and can combine several columns — here, summing columns `a` and `b` into a new value per row.

Common pitfall `apply` is flexible but **slow**: it runs your Python function once per row, which on a million rows is a million function calls. Whenever a built-in **vectorised** operation exists, prefer it — `df["price"] * 100` is both shorter and far faster than `df["price"].apply(lambda x: x * 100)`. Reach for `apply` only for logic that genuinely cannot be expressed with pandas' own operations.

Remark A `lambda` is convenient but anonymous; if the logic grows beyond a line or you want to reuse it, write a named `def` function and pass it by name — `df["price"].apply(to_cents)`

— exactly as you learned to define and call functions before. `apply` accepts any function, lambda or not.

8 The End-to-End Pipeline

Every technique so far is a single move; real work **chains** them, from a raw file to a clean, summarised result. This chapter assembles the whole arc — it is the course in one script.

Definition 11 (The four stages)

1. **Load** the raw file into a DataFrame.
2. **Inspect** it — shape, types, missing values — to learn what you are dealing with.
3. **Clean** it — handle missing values and obvious errors, recording each decision.
4. **Summarise** it — group and aggregate into the table that answers the question.

Worked example — Raw sales file to per-city summary.

```
import pandas as pd

# 1. Load
df = pd.read_csv("sales.csv")

# 2. Inspect
print(df.shape)            # how big?
print(df.info())           # types and where values are missing
print(df.describe())       # distribution; check min/max for errors

# 3. Clean
df = df.dropna(subset=["sales"])               # a row with no sales is useless
df["city"] = df["city"].fillna("Unknown")      # keep the row, flag the gap
df = df[df["sales"] >= 0]                       # drop impossible negatives

# 4. Summarise
summary = (
    df.groupby("city")["sales"]
      .agg(["sum", "mean", "count"])
      .reset_index()
      .sort_values("sum", ascending=False)
)
print(summary)
```

Read the cleaning block as a series of **defensible decisions**, not mechanical steps: rows with no sales figure are dropped because they answer no question; missing cities are kept but flagged so the row still counts toward totals; negative sales are removed as impossible. Each line is a judgement you could explain to a sceptic — which is exactly the standard the next chapter holds you to.

Remark Notice the parentheses around the summary block: wrapping a chain of calls in `( … )` lets you put each step (`.groupby`, `.agg`, `.reset_index`, `.sort_values`) on its own line. The chain reads top to bottom like a recipe, and you can comment or comment-out any single step. Prefer this to one unreadable line or to a thicket of intermediate variables.

Common pitfall Order matters in a pipeline. If you `groupby("city")` **before** filling missing cities, those rows vanish from the totals (recall `groupby` drops `NaN` keys); if you compute a mean **before** removing impossible values, the outliers poison it. Inspect first, clean second, summarise last — and within cleaning, fix the errors that would distort a later step before you take that step.

9 Descriptive Reporting

A summary table is not the finish line. A manager does not want a DataFrame; they want to know **what it means**. The final, and most undervalued, skill of this course is turning a table of numbers into a short paragraph a human can act on.

Definition 12 (A descriptive report) A descriptive report states, in plain language, what the summary shows: which groups are largest and smallest, where the values concentrate, and any notable gaps or outliers. It **describes what the data says** and does not claim **why** — causes lie beyond what a descriptive summary can support.

Worked example — From table to paragraph. Given the per-city summary (Paris: sum 152 000, mean 305, count 498; Lyon: sum 88 000, mean 210, count 419; Unknown: sum 12 000, mean 240, count 50), a good report reads:

> "Paris dominates total sales (€152 000), driven both by the largest order count (498) and the highest average order value (€305). Lyon, despite a similar number of orders (419), contributes far less (€88 000) because its average order is a third smaller (€210). The 50 orders with an unknown city (€12 000) are too few to affect the overall picture but should be investigated, as a missing-city rate of this size may hide a data-collection problem."

Every sentence points back to a number in the table, contrasts groups, and stops short of inventing a cause. That is the genre.

Common pitfall The cardinal sin of reporting is sliding from description to unfounded **causation**: "Paris sells more **because** its customers are wealthier" is not in the data — you measured sales, not wealth. State what the numbers show; flag what would need further data to explain. Confident over-claiming from a descriptive table is how analyses lose their credibility.

Common pitfall Equally, do not bury the finding under every number. A report is not the table read aloud; it is the **two or three things that matter** — the biggest group, the surprising contrast, the worrying gap — stated plainly. If your paragraph mentions every cell, it has interpreted none.

Course takeaways. A DataFrame is Series sharing an index; everything else is operations on that table. **Inspect** before you compute (`shape`, `info`, `describe`) and check types and missing counts. **Select** with `loc` (labels) and `iloc` (positions), and **filter** with parenthesised

boolean conditions joined by `&`/`|`. **Handle missing values** by a justified drop or impute, and say which and why. **Summarise** with split–apply–combine via `groupby`. **Combine** tables by concatenating (same shape) or merging (shared key), choosing the join type by which rows must survive. **Reshape** between wide and long with `pivot` and `melt`. Use `apply` only when no vectorised operation will do. Chain it all into a load → inspect → clean → summarise pipeline, and close with a descriptive paragraph that says what the data shows without claiming why.

Exercises

1. Load a CSV into a DataFrame and report its `shape`, `dtypes`, and the output of `describe`. Identify one column whose `count` is below the row total and say how many values are missing.
2. From the same table, select a single column as a Series and the same column as a one-column DataFrame, and explain the difference between the two results.
3. Use `loc` and `iloc` to retrieve the same cell two ways, then change the index (e.g. set a name column as the index) and show a case where `loc` and `iloc` now disagree.
4. Select all rows where one numeric column exceeds its own mean **and** a categorical column equals a chosen value, using a single boolean expression with correct parentheses.
5. Count the missing values in each column. Then impute one numeric column and one categorical column, each with a different strategy, and write one sentence justifying each choice.
6. Group the data by a categorical column and compute the sum, mean, and count of a numeric column in one call. Then group by **two** columns and describe what each row of the result represents.
7. Merge two DataFrames with an inner join and again with a left join. Report the row count of each result and explain, in terms of unmatched keys, why they differ.
8. Reshape a DataFrame from wide to long with `melt`, then back to wide with `pivot`, and confirm you recover the original. State one task that is easier in each layout.
9. Add a new column computed with `apply` and a `lambda`, then rewrite the same computation as a vectorised operation and explain why the vectorised version is preferable.
10. Write a complete pipeline on a real CSV — load, inspect, clean (recording each decision as a comment), and summarise with a grouped aggregation — and finish with a paragraph interpreting the findings without claiming causes.